s

Technical University of Denmark

Written examination date: May 23, 2025

Course title: Computer Science Modelling

Course number: 02141

Aids allowed: All aid

Exam duration: 4 hours

Weighting: 7-step scale

RULES & RECOMMENDATIONS:

R1 You must hand-in your submission **digitally**.

R2 Submissions must be hand-in as follows:

- **Main Document**: Upload here your PDF. The name of the file must be **main.pdf**.
- **Attachments**: Upload here the **files** folder zipped into a file called **files.zip**. The folder should have the same subfolders and files as you downloaded. The files, of course, should contain your solutions. IMPORTANT: remember to upload the **files** folder, not only the sub-folders.

R3 You are allowed to hand-in paper sheets in addition to your digital submission. If you submit some answers on paper, you must clearly indicate in your digital version (at the respective position for the answer) that the solution is provided on paper and on which page.

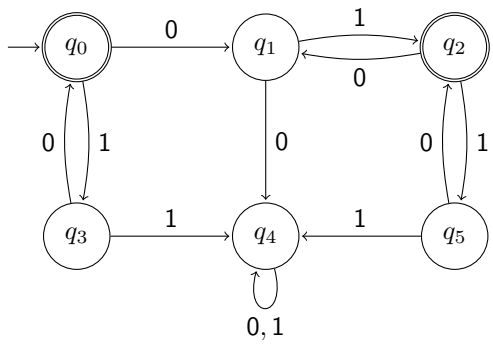
R4 Some exercises require that you provide files that can be used with the tools seen in the course (fm4fun, inspectify, chip, moka). In those cases, your digital submission must follow the precise naming of the files as provided. So, please do not rename the file names. Wrong file names and screenshots are not valid submissions. It is your responsibility to follow carefully the formatting requirements of the files. For example, GCL code with parsing errors will not be accepted.

R5 You must clearly indicate on each page your student number, and which exercise you attempt to solve.

R6 We recommend that you first skim all tasks before deciding in which order you want to solve them.

Task 1 – Formal Languages

Alice designs the following DFA A over $\Sigma = \{0, 1\}$



Question 1a: Since Bob has hard time understanding DFAs, Alices decides to try minimizing A as a new equivalent DFA A'. Help Alice obtaining A'. Use the minimisation method seen in the course. You do not need to show the intermediate steps. You just need to provide A' as a transition diagram in the PDF. Each state of A' should be labeled with subsets of states of A.

Question 1b: Eve prefers regular expressions over DFA. She designs the following regular expression E: $\mathtt{1}^*(\mathtt{01})^*$.

Alice and Eve meet to discuss how their models relate to each other. Help them by filling the table below. In the empty cell corresponding to language L provide a short-as-possible word that belongs to L. If no such word exists, write `NONE`.

Language L	Shortest word
$\mathcal{L}(\mathsf{A'}) \cap \mathcal{L}(\mathsf{E})$	
$\mathcal{L}(\mathsf{A'}) \cup \mathcal{L}(\mathsf{E})$	
$\mathcal{L}(\mathsf{A'}) \setminus \mathcal{L}(\mathsf{E})$	
$\mathcal{L}(\mathsf{E}) \setminus \mathcal{L}(\mathsf{A'})$	
$\overline{\mathcal{L}(\mathsf{A})} \cup \mathcal{L}(\mathsf{E})$	
$\overline{\mathcal{L}(\mathsf{A})} \cap \overline{\mathcal{L}(\mathsf{E})}$	
$\mathcal{L}(\mathsf{A})^* \cup \mathcal{L}(\mathsf{E})^*$	

Provide the table in text format in file **task1b.txt**, with one line per row. For example, if you think that the answer in all cases is 0101010 write

```
0101010
0101010
0101010
0101010
0101010
0101010
0101010
0101010
```

You can denote the empty string just a blankline.

Question 1c: George is a fan of grammars. He has discovered a grammar G in a book on modal logics, which has modalities to reason about events that occurr *once* ($\mathbf{O}$) is and *historically* ($\mathbf{H}$). The grammar G consists of the following productions:

$$
\begin{array}{rcll}
F & \rightarrow & A & (1) \\
 & | & F \wedge F & (2) \\
 & | & A\ R\ N & (3) \\
 & | & \mathbf{O}F & (4) \\
 & | & \mathbf{H}F & (5) \\
\mathbf{HH}F & \rightarrow & \mathbf{H}F & (6) \\
A & \rightarrow & \mathsf{a} & (7) \\
 & | & \mathsf{b} & (8) \\
N & \rightarrow & \mathsf{1} & (9) \\
 & | & \mathsf{2} & (10) \\
R & \rightarrow & \leq & (11) \\
 & | & \geq & (12)
\end{array}
$$

where

- The non-terminal symbols are F, A, N and R;
- The terminal symbols are $\wedge$, $\mathbf{O}$, $\mathbf{H}$, a, b, $\mathsf{1}$, $\mathsf{2}$, $\leq$ and $\geq$;
- The starting symbol is F;
- The numbers of the right indicate the number of the production.

George wants to show the grammar G to Alice and Eve, who are familiar with context-free grammars. But...

... is grammar G context-free? Provide in file **task1c.txt** the smallest set of productions that need to be removed to make G context-free. Write one number per line. For example if you think that the first 3 production should be removed to make the grammar context-free write

```
1
2
3
```

If you think that the grammar is already context-free, leave the file empty.

Question 1d: Alice wants to implement a parser for the grammar G' obtained from G after removing the productions identified in (c).

She asks George a relevant question: Does the grammar G have ambiguities? Help Bob answering the question. If your answer is YES, provide in file **task1d.txt** pairs of productions that yield ambiguities. For example, if you think that there is an ambiguity stemming from some combination of the productions 1 and 2, and another one stemming from some combination of 1 and 3 write

```
1,2
1,3
```

If you think there is no ambiguity, leave the file empty.

In addition to the file, please provide a short explanation in the PDF. If you think that there are no ambiguities, please write 1-3 lines explaining why. If you think that there are ambiguities, select one and showcase an example of the ambiguity as we did in the lectures. If you need to write text, limit yourself to 1-3 lines.

Question 1e: We now want to help Bob address question (d) more in depth. If your answer to (d) is NO, expand your explanation in (d) by providing a formal proof. If your answer to (d) is YES, fix the ambiguity you chose to demonstrate in (d) by a new grammar that removes said ambiguity. Explain the changes in 1-3 lines.

Task 2 – Programming Language Semantics

Alice wants to run GCL programs efficiently. For that purpose she wants to build compiler that translates GCL code into a bytecode language. She has been told that bytecode languages allow at most one binary operation in each variable update. Following the recommendation, she prepares the following function bin() that transforms GCL programs into GCL programs:

```
        bin(skip)  =  skip
        bin(v:=u)  =  v:=u
     bin(v:=a1@u)  =  bin(v:=a1) ; v:=v@u
     bin(v:=u@a1)  =  bin(v:=a1) ; v:=u@v
    bin(v:=a1@a2)  =  bin(v:=a1) ; bin(v:=v@a2)
      bin(v:=-a1)  =  bin(v:=-1*a1)
     bin(v:=(a1))  =  bin(v:=a1)
     bin(c1 ; c2)  =  bin(c1) ; bin(c2)
    bin(if gc fi)  =  if bin(gc) fi
    bin(do gc od)  =  do bin(gc) od
     bin(p -> c1)  =  p -> bin(c1)
  bin(gc1 [] gc2)  =  bin(gc1) [] bin(gc2)
```

where

- `v` is any variable,
- `u` is an integer number or a variable,
- `a1`, `a2` are arithmetic expressions,
- `@` is any of the binary arithmetic expressions of GCL (`+,*,/,-,...`),
- `c1`, `c2` are commands,
- `p` is a boolean predicate, and
- `gc`, `gc1`, `gc2` are guarded commands.

Question 2a: Provide in file **task2a.txt** the GCL program that results from applying function bin() to the GCL program below.

```
x := a + b * c - d
```

where `x`, `a`, `b`, `c`, and `d` are variables.

Question 2b: Bob suspects that there is something wrong in function bin(). Confirm Bob's suspects by providing the following:

- a memory σ_1 in file **task2b1.txt**;
- a memory σ_2 in file **task2b2.txt**;
- a memory σ_3 in file **task2b3.txt**;
- a deterministic GCL program C in file **task2b4.txt**;
- the transformed program bin(C) in file **task2b5.txt**;

such that

1. C has a complete execution sequence starting with the initial memory σ_1 and ending in memory σ_2;
2. bin(`C`) has a complete execution sequence starting with the initial memory σ_1 and ending in memory σ_3;
3. $\sigma_2 \neq \sigma_3$.

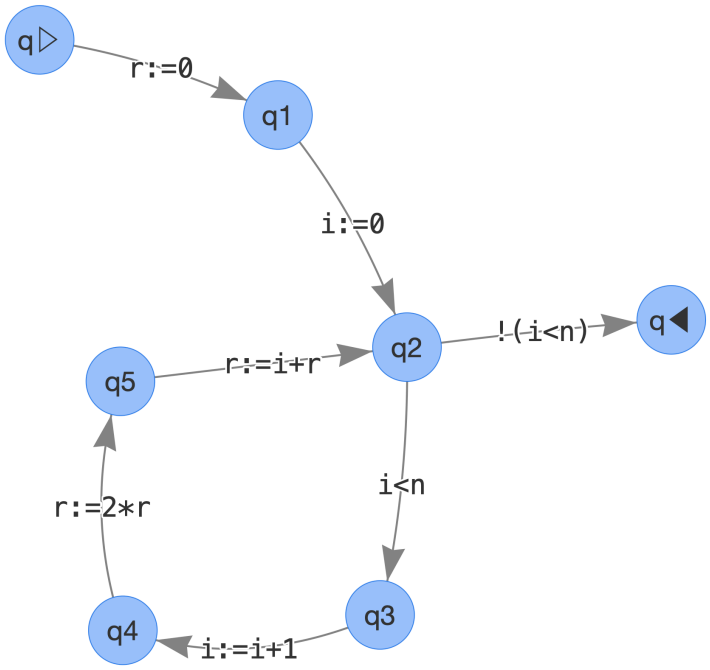
The memories σ_1, σ_2 and σ_3 must be provided as GCL boolean predicates (with no short-circuit operations). For example, the memory $x \mapsto 1, y \mapsto 2, z \mapsto 3$ must be provided as

```
x=1 & y=2 & z=3
```

HINT: Loops are not needed. Aim at the smallest possible example.

In addition to the files, please explain in 1-3 lines in the PDF what is wrong with function bin().

Question 2c: Alice finds the following Program Graph on Bob's whiteboard



It is the program graph bin(C) of some command C. Your task is to help Alice discover C. Provide C in file **task2c.txt**.

Question 2d: Alice and Bob engage in a discussion about properties of GCL commands, program graphs and the function bin(). Help them by answering with `True` or `False` the following questions :

1. For all commands C, if C is evolving then bin(C) is evolving.
2. For all commands C, if C constitutes a deterministic system then bin(C) constitutes a deterministic system.
3. For all commands C it holds that bin(C) = bin(bin(C)).
4. There exist commands C and C' such that bin(C) = bin(C').
5. For all commands C, the Program Graph of C is the same as the Program Graph of bin(C).
6. For all commands C, the Program Graph of C has at least the same number of nodes than the Program Graph of bin(C).
7. Given a program graph PG, if there exists a command C whose program graph is precisely PG, then there exists also some command C' such that PG is the Program Graph of bin(C').
8. For all commands C and C', if C, bin(C), and bin(C') have the same complete execution sequences, then $C = C'$.

Provide your answers in file **task2d.txt**, writing one answer (`True`/`False`) per line. For example, if you believe only the even answers are correct, just write

```
False
True
False
True
False
True
False
True
```

In addition to the file, please select one `True` statement and one `False` statement and explain (in 1-3 lines each) in the PDF your answer.

Question 2e: Alice wrote the below theorem and proof as part of her attempt to show the correctness of bin().

Theorem 1. *For all memories σ, all u (either integers or variables in $dom(\sigma)$), and all arithmetic expressions a, a_1, a_2, the following properties hold:*

$$\begin{array}{rcll} \mathcal{A}[\![u]\!]\sigma & = & \mathcal{A}[\![u]\!]\sigma & (P1) \\ \mathcal{A}[\![a@u]\!]\sigma & = & \mathcal{A}[\![x@u]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & (P2) \\ \mathcal{A}[\![u@a]\!]\sigma & = & \mathcal{A}[\![u@x]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & (P3) \\ \mathcal{A}[\![a_1@a_2]\!]\sigma & = & \mathcal{A}[\![x@a_2]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma] & (P4) \end{array}$$

Proof by cases:

Case P1 $\mathcal{A}[\![u]\!]\sigma \quad = \quad \mathcal{A}[\![u]\!]\sigma$ (step 1)

Case P2 (case P3 is similar and skipped)

$$\begin{array}{rll} & \mathcal{A}[\![x@u]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & \\ = & \mathcal{A}[\![x]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma]@\mathcal{A}[\![u]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & \text{(step 2)} \\ = & \mathcal{A}[\![x]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma]@\mathcal{A}[\![u]\!]\sigma & \text{(step 3)} \\ = & \mathcal{A}[\![a]\!]\sigma@\mathcal{A}[\![u]\!]\sigma & \text{(step 4)} \\ = & \mathcal{A}[\![a@u]\!]\sigma & \text{(step 5)} \end{array}$$

Case P4

$$\begin{array}{rll} & \mathcal{A}[\![x@a_2]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma] & \\ = & \mathcal{A}[\![x]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma]@\mathcal{A}[\![a_2]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma] & \text{(step 6)} \\ = & \mathcal{A}[\![a_1]\!]\sigma@\mathcal{A}[\![a_2]\!]\sigma & \text{(step 7)} \\ = & \mathcal{A}[\![a_1@a_2]\!]\sigma & \text{(step 8)} \end{array}$$

Help Alice determining which of the 8 steps in the proof is correct or incorrect. A proof step is correct if the equality stated in the same line holds.

For simplicity, you can assume that all expressions are defined and that Alice has been using a simplified version of the definition of $\mathcal{A}$, e.g.

$$\mathcal{A}[\![a_1 + a_2]\!]\sigma = \mathcal{A}[\![a_1]\!]\sigma + \mathcal{A}[\![a_2]\!]\sigma$$

Provide your answers in file **task2e.txt**, writing one answer (`Correct`/`Incorrect`) per line. For example, if you believe only the even steps are correct, just write

```
Incorrect
Correct
Incorrect
Correct
Incorrect
Correct
Incorrect
Correct
```

In addition to the file, please select one correct step and one incorrect step and explain (in 1-3 lines each) in the PDF your answer.

Task 3 – Verification & Analysis

Alice and her friends want to study the security properties of the below program.

```
r := 1;
i := 0;
do
  i < n ->
    r := e * r;
    i := i + 1
od
```

Question 3a: All the variables of the program have to be considered as secret. However, Alice has learned that malicious people like Trent could learn something about variables by measuring power consumption of a program. Alice is told that energy peaks can be caused by variable updates. She is concerned about leaking information about the secret variables in the program. She decides to introduce a variable `power` to keep track of power consumption and she instruments her code as follows:

```
power := 0;
r := 1;
i := 0;
power := power + 2;
do
  i < n ->
    r := e * r;
    power := power + 1;
    i := i + 1 ;
    power := power + 1
od
```

Alice has attended 02141 and wants to start with something simple: information flow analysis as provided by fm4fun and inspectify. Help Alice by providing in file **task3a.txt** the list of variables that flow into variable `power`. The file should contain one variable per line.

Question 3b: In addition to information flow analysis based on fm4fun, Alice also wants to use non-interference and verification to determine if the program is secure. She wants to check if Trent can learn something about variable secret `e` through public variable `power`. She wants to use the technique seen in the course, where non-interference properties are formalized as a verification problem on a duplicated program. Help Alice by providing:

- The pre-condition P (without curly brackets) in file **task3b1.txt**
- The duplicated program C in file **task3b2.txt**
- The post-condition Q (without curly brackets) in file **task3b3.txt**

such the triple $\{P\}C\{Q\}$ formalizes the desired non-interference property. In addition to predicates related to non-interference, P should exclude cases where e is non-positive and n is negative.

When renaming variables, simply append `1` and `2` for each of the copies. For example, `x:=y+z` in the first copy should become `x1:=y1+z1`.

In addition to the files, please select explain (in 1-3 lines each) in the PDF what you would conclude from the analysis of the triple.

Question 3c: Bob notices Alices's solution to (a) and (b). He suggests that power consumption depends on `n` in a very precise manner. He proposes the below Hoare-Floyd Triple to formalize his idea.

```
{ 0 <= n & 1 < e }
power := 0;
r := 1;
i := 0;
power := power + 2;
do  [ // some invariant ]
  i < n ->
    r := e * r;
    power := power + 1;
    i := i + 1 ;
    power := power + 1
od
{ r = exp(e, n) & power = 2+2*n }
```

Help Bob to confirm his idea by providing in file **task3c.txt** the above annotated command, where you replace the comment about the invariant with an actual invariant that makes the triple verify.

In addition to the file, please select explain (in 1-3 lines each) in the PDF what you would conclude from the analysis of the triple.

Question 3d: Charlize joins the room. She is an expert in computer architectures, and she is concerned about how power consumption is accounted for when dealing with multiplications. She knows chips where multiplications involve multiple updates. She proposes to modify the instrumented code as follows.

```
power := 0;
r := 1;
i := 0;
power := power + 2;
do
  i < n ->
    r := e * r;
    power := power + e;
    i := i + 1 ;
    power := power + 1
od
```

This changes of course the post-condition from (c). Help Charlize find the new post-condition (without curly brackets) in file **task3d.txt** which precisely determines the value of `power` at the end of the program. In addition to the files, please explain (in 1-3 lines each) in the PDF what you would conclude from the analysis of the new post-condition.

Question 3e: Alice and her friends have a final discussion on the security of the program. Help them by answering with `True` or `False` the following questions:

1. The verification in (b) was not necessary, in the sense that we could have inferred whether Trent can learn something about variable `e` from the information flow analysis in (a).
2. From the verification in (b) we can conclude, for each secret variable, whether Trent can learn somehting about them, just by observing `power`.
3. If we perform information flow analysis on the program in (d) we can precisely determine whether or not Trent can learn something about variable `e`.
4. To determine how `power` depends on the variables of the program in (c) and in (d) we need to know what the program computes, i.e. we need to have the predicate `r = exp(e, n)` as part of the postcondition.
5. If we restrict to inputs where $n <= m$, we can make the program in (c) have constant power consumption by incrementing `power` by $2 * m - 2 * i$ after the loop.
6. If we restrict to inputs where $n <= m$, we can make the program in (c) have constant power consumption by incrementing `power` by $2 * m - 2 * i$ after the loop.
7. If we restrict to inputs where $n <= m$, we can make the program in (d) have constant power consumption by incrementing `power` by $2 * m - 2 * i$ after the loop.

Provide your answers in file **task3e.txt**, writing one answer (`True`/`False`) per line. For example, if you believe only the even answers are correct, just write

```
False
True
False
True
False
True
False
True
```

In addition to the file, please select one `True` statement and one `False` statement and explain (in 1-3 lines each) in the PDF your answer.

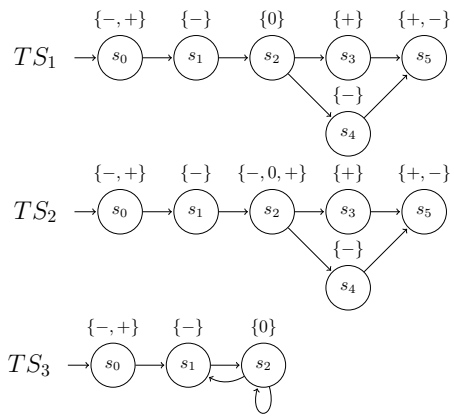
Task 4 – Model Checking & Sign Analysis

Alice wants to use model checking to reason about signs. For this purpose, she decides to build transitions systems out of sign analysis assignments.

More precisely, given a sign analysis assignment $\mathbf{A}$ for a program graph $\mathbf{P}$ with set of nodes Q and set of edges E, and a variable x, we can build a transition system $TS_{\mathbf{P},\mathbf{A},x}$ as follows:

- $S = Q$
- $I = \{s_0\}$, where s_0 is the initial node of Q.
- $\longrightarrow = \{(q, q') \mid (q, a, q') \in E\}$
- $\mathbf{AP} = \mathsf{PowerSet}(\{-, 0, +\})$
- $\mathbf{L}(s) = \mathbf{A}(s)(x)$

Question 4a: Bob tries to follow Alices' idea and provides the following transition systems with $AP = \{-, 0, +\}$



For each of the transition systems, determine if there exist $\mathbf{P}$, $\mathbf{A}$ and x such that $TS_i = TS_{\mathbf{P},\mathbf{A},x}$.

Provide your answer in text format in file **task4a.txt**, writing `YES` or `NO`. One transition system per line. For example if you think, the answer is positive for the last case only, write.

```
NO
NO
YES
```

In addition, write in the PDF an explanation in 1-3 lines for each case.

Question 4b: Alice writes LTL properties to reason about signs in the transitions systems of (a). Help Alice by filling the table below. In the empty cell corresponding to an LTL property ϕ and a transition system TS_i (where $i \in \{2, 3\}$) provide an answer `YES` if $TS_i \models \phi$, and `NO` if $TS_i \not\models \phi$.

Property	TS_2	TS_3
$\mathbf{X}-$		
$\mathbf{X}+$		
$\mathbf{F}(\neg - \wedge \neg +)$		
$\mathbf{G}+$		
$\mathbf{GF}(- \wedge +)$		

Provide the table in text format in file **task4b.txt**, with just colons to separate columns and one line per row. For example, if you think that are all `YES` for all cases, write:

```
YES,YES
YES,YES
YES,YES
YES,YES
YES,YES
```

Question 4c: Bob has written some properties about the signs of a specific variable `x` in a GCL programing natural language. He needs help to formalize them in LTL. Provide the formalisation of the below properties in LTL. As the set of atomic propositions, assume that there is only one proposition `x < 0` that holds only in states where `x` is negative. You are not allowed to use any other proposition.

The properties are:

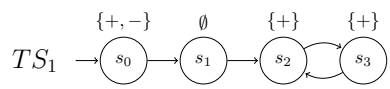
1. `x` cannot be negative for more than three states in a row.
2. `x` is initially negative for zero or more states, and then becomes non-negative and does ever enter a state where it becomes negative.

Provide your answer in file **task4c.txt** with one line per formula. Formulas have to be in the LTL format of `moka`. Examples of solutions that are well formatted:

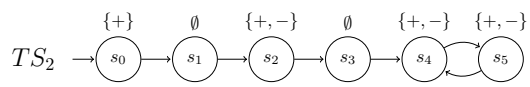
```
G x<0
F ! x<0
```

Question 4d: Dave likes using LTL for sign analysis. He designs the below transition systems TS_i together with their corresponding LTL properties ϕ_i.

1. $\phi_1 = +\mathbf{U}(\mathbf{X}-)$



2. $\phi_2 = \mathbf{G}(+ \rightarrow \mathbf{XXX}+)$



Unfortunately, the properties do not hold. You need to help Dave repair the transition systems by changing the labels of some states. In particular, we want to change just one state at most: the state s_j with smallest j such that with the new label $L(s_j)$, TS_i satisfies ϕ_i. The new set $L(s_j)$ should be as small as possible.

Provide your answer in file **task4d.txt** indicating the state and the labels as a list (first the state, then the labels in alphabetical order). Use one line per case. For example, if you think that in both cases, state s_1 should be relabelled with $\{0, -\}$ write

```
s1,0,-
s1,0,-
```

Question 4e: Eve read about *past LTL*, which is a variant of LTL over finite traces, with modalities $\mathbf{Y}$ (*yesterday*) and $\mathbf{H}$ (*historically*).

Formally, a finite trace $\pi = a_0 a_1 \ldots a_n$ is accepted by a past LTL formula ϕ iff $w, n \models \phi$, where the relation $\models$ is formalised as follows:

$$
\begin{array}{lcll}
\pi, i & \models & a & \text{if } \pi[i] = a \\
\pi, i & \models & \phi_1 \wedge \phi_2 & \text{if } \pi, i \models \phi_1 \text{ and } \pi, i \models \phi_2 \\
\pi, i & \models & \phi_1 \vee \phi_2 & \text{if } \pi, i \models \phi_1 \text{ or } \pi, i \models \phi_2 \\
\pi, i & \models & \phi_1 \rightarrow \phi_2 & \text{if } \pi, i \models \phi_1 \text{ implies } \pi, i \models \phi_2 \\
\pi, i & \models & \neg\phi & \text{if } \pi, i \not\models \phi \\
\pi, i & \models & \mathbf{Y}\phi & \text{if } i > 0 \text{ and } \pi, i-1 \models \phi \\
\pi, i & \models & \mathbf{H}\phi & \text{if } \forall j \leq i . \pi, j \models \phi
\end{array}
$$

Help Eve fill the below table by finding for each property a finite trace that satisfies it. The traces must be minimal:

- as short as possible
- if several label sets for a state are possible select the smallest set
- if two sets of labels of equal size are possible, select the smallest option using lexicographical order with $- < 0 < +$. For example $\{0, -\}$ comes before $\{0, +\}$.

Property	Trace
$a \wedge \mathbf{Y}b$	
$\mathbf{H}b$	
$b \wedge \mathbf{H}(b \rightarrow \mathbf{Y}a)$	

Provide your answer in file **task4e.txt**. For example, if you think that in all cases the traces should consist of just three states, the first two having `a,b` as labels and the last state having the empty set as a label, write

```
{a,b},{a,b},{}
{a,b},{a,b},{}
{a,b},{a,b},{}
```

You can denote a trace of length 0 with just a blank line.